

Agile Software Development with Distributed Teams: Agility, Distribution and Trust

Siva Dorairaj
School of Engineering and
Computer Science
Victoria University of Wellington,
Wellington, New Zealand
siva.dorairaj@ecs.vuw.ac.nz

James Noble
School of Engineering and
Computer Science
Victoria University of Wellington,
Wellington, New Zealand
james.noble@ecs.vuw.ac.nz

Abstract—Trust among team members is imperative for blending agility with distributed software development. Little is known about how team members who are dispersed across different geographical locations, time zones and cultures can build trust while working together in Agile software development projects. Through a Grounded Theory study that involved 55 participants from 38 different software companies in the USA, India and Australia, we investigate the key concerns of distributed teams in Agile software development. We found that trust among members of a distributed team is important for bridging spatial, temporal and socio-cultural distances in order for them to work together as one team. In this paper, we present techniques for building trust in Agile software development with distributed teams.

Keywords—Trust, Agile Methods, Grounded Theory, Distributed Team, Global Software Development

I. INTRODUCTION

Trust is an important factor that affects the performance of a software development team [10], [52]. Trust among team members is imperative for blending agility with distributed teams and achieving success in the software development projects [52], [54], [58]. Without trust, a distributed team faces difficulties to develop cohesion among its members, which in turn, affects performance of the team [38], [52]. A substantial effort is required to build trust between the team members who are distributed across different countries, in different continents. Understanding the techniques for building trust is important for developing high performance, cohesive teams [5], [45], [69]. There is, however, little empirical studies on the techniques for building trust in Agile software development with distributed teams [33], [51]. Several studies [38], [67], [70] point out that building trust in distributed teams in general can be difficult.

Through a Grounded Theory study that involved 55 participants from 38 different software companies in the USA, India, and Australia, we investigate the key concerns of distributed teams in Agile software development. The initial analysis gave rise to a number of key concerns for distributed teams such as bridging cultural differences, improving communication, managing knowledge and building trust. We then commenced in-depth investigation on these emergent key concerns. In this paper, we describe the techniques for building and sustaining trust in Agile software development with distributed teams.

II. BACKGROUND: AGILITY AND TRUST

Trust has been studied in many fields such as organisational management [49], psychology [59], computer science [47], social science [20] and medical [32]. Mayer et al. [49] define trust as “the willingness of a party to be vulnerable to the actions of another party based on the expectation that the other will perform a particular action important to the trustor, irrespective of the ability to monitor or control that other party”. Mayer et al. [49] characterize trust into three kinds: ability, benevolence and integrity. Ability refers to the aptitude and skills that enable an individual to be perceived as competent by team members. Benevolence refers to the extent to which an individual is believed to be willing to do good to team members beyond personal profit motive. Integrity refers to the extent to which an individual is believed to adhere to a set of principles thought to make the individual dependable and reliable. An individual is deemed trustworthy when the ability, benevolence and integrity are all perceived to be high [49]. In this study, we use Mayer et al.’s [49] definition of trust because it includes both the belief and behavioural components of trust, and it also incorporates the notion that both trustor and target are necessary for trust to be present. In general, a team member is deemed trustworthy when the team member is willing to respond appropriately to the need of the team and use existing skills and knowledge for the benefit of the team and the overall success of the project.

Agile teams are cross-functional and self-organizing in nature [34], [36], [64]. A cross-functional Agile team comprises everybody involved in the development of a project, including people from the business area, project management, development, testing, quality assurance, and technical support. The members of a self-organizing team “manage their workload, shift work among themselves based on need and best fit, and participate in team decision making” [34]. The Agile Manifesto [18] asserts that the best architectures, requirements and designs emerge from self-organizing teams. In this study, we define an *Agile team* as a cross-functional, self-organising, high-performance team that carries out software development activities inline with the principles behind the Agile Manifesto in order to provide business value to customer. A classic Agile team is a small, collocated team where the customer is an integral part of the team [36], [48].

The emergence of self-organizing teams has increased the importance of trust in software development activities [36], [50]. Managers must trust that members of an Agile team will deliver business value to customer [51]. Members of an Agile team must trust that every member of the team is competent and knowledgeable to design, develop and deliver valuable software for customers [50], [54]. An Agile team that exhibits a high degree of trust experiences improved communication and increased team performance [52]. Conversely, trust declines when team members fail to fulfill mutual obligations or the perception of obligations differs between some team members and the individual responsible for meeting them [56]. Lack of trust within the team affects how effectively individuals contribute to teams, reduces overall team performance, increases cycle time, creates higher costs and impacts product quality [7]. It is not surprising that lack of trust was one of the reasons for customers not actively involved in the implementation of Agile software development with distributed teams [42]. In the absence of trust, the members of a team will doubt whether the team as a whole can deliver business value to customer [60].

While trust is crucial for the success of a project, several existing studies [5], [40], [43], [52], [55], [61] point out that building trust amongst members of a distributed team is difficult because an individual involved in the project often have little or no prior experience working together with other individuals in the team. Trust is also considered to be very fragile in distributed teams due to the spatial, temporal, and socio-cultural distances [37], [38], [41], [58]. In this study, we use the term *distributed team* to refer to a software development team from the *same* organisation where the members of the team are dispersed across different countries, in different continents. There is a paucity of studies into the techniques for building trust amongst members of a distributed team, particularly in Agile software development. In this paper, we describe the techniques for building trust in Agile software development with distributed teams.

III. RESEARCH METHOD

Grounded Theory (GT) is a systematic research method that emphasises the generation of theory derived from systematic and rigorous analysis of data. GT was jointly developed and first articulated by Barney G. Glaser and Anslem L. Strauss [30]. GT researchers gather data, particularly qualitative data from interviews and observations, and systematically discover a *theory* derived directly from the data [35], [48]. GT is intended neither to answer a specific research question nor to test an existing hypothesis, but rather to focus on discovery of a substantive theory regarding the research phenomenon within its context [3], [8], [57].

With the aim to generate a theory that is emergent from empirical data rather than from inferences or existing theories, we chose classic GT [23], [24], [25], [26], [27], [28], [29] as our research method for two main reasons. Firstly, GT is suitable to be used in areas that are under-explored or where a new perspective might be beneficial [1], [62], and literature

on Agile software development with distributed teams is still scarce [17], [41]. Secondly, GT allows researchers to study social interactions and the behaviour of people [30], and Agile methods focus on people and their interactions in software development teams [63].

In the recent years, GT is successfully being used to study social aspects of Agile teams [19], [36], [48], [58], [68]. Our study started with a general area of interest – Agile software development with distributed teams – rather than beginning with a specific research question [9], [26]. This does not mean that there is no specific problem for the research, but rather formulating a research question before commencing a GT study can lead to preconceived ideas of the research phenomenon [31], [35], [53]. Glaser [25], [26] asserts that the problem and its key concerns will emerge in the initial stages of data analysis.

A. Context

In this study, we interviewed with 55 participants from 38 different software companies in the USA, India and Australia. Due to privacy and ethical consideration, we will only identify the participants using the codes P1 to P55. Table 1 shows the participant and project details.

Team distribution varied between 2 and 4 locations across different countries, in different continents. Project domain includes e-commerce, financial services, Internet services and telecommunication. Project iteration varied between 2 and 4 weeks, and project duration varied between 6 and 36 months (some projects were still ongoing during data collection). Although projects started with a small team size of 6 to 12 members, some teams had scaled through to a large team size in order to accommodate the increasing complexity of the projects.

Scrum or a combination of Scrum and XP were used in the projects through adoption of fundamental practices such as small releases, continuous integration, estimation of user stories, iteration planning, daily standup, release planning, sprint reviews and sprint retrospectives. In order to get a rounded perspective of the research phenomenon, we interviewed participants from different roles in Agile software development: Developer, Quality Analyst, Business Analyst, Scrum Master, Agile Coach, Product Owner, and Senior Management (e.g. Vice President of Delivery, Human Resource Manager, and Director of Technology). Several participants were notably active in communities of practice (e.g. Agile Project Management Group, Agile Software Community of India, Agile and Lean Software Development) where they have been participating in forums, authoring articles based on subject matter expertise, and speaking at local events and international conferences.

B. Data Collection and Analysis

Data collection in GT is guided through *theoretical sampling* whereby the researcher jointly collects, codes, and analyzes his data and decides what data to collect next and where to find them, in order to develop his theory as it emerges [23].

TABLE I
SUMMARY OF PARTICIPANTS, TEAMS, AND PROJECTS. (AGILE ROLE: SCRUM MASTER (SM), AGILE COACH (AC), DEVELOPER (DEV), BUSINESS ANALYST (BA), QUALITY ANALYST (QA), PRODUCT OWNER (PO), SENIOR MANAGER (MGT))

PARTICIPANT			TEAM		PROJECT			
Code	Location	Agile Role	Distribution	Size	Domain	Agile Methods	Duration (months)	Sprint (weeks)
P1	USA	DEV	N.America-Asia	8 to 10	Financial Services	Scrum	10	2
P2	USA	AC	N.America-Asia	12 to 14	E-Commerce	Scrum/XP	12	2
P3	USA	SM	N.America-Europe-Asia	10	Mobile Application	Scrum	8	3
P4	USA	AC	N.America-Asia	10	Online Trading	Scrum/XP	8	2
P5	USA	AC	N.America-Asia	8	Internet Media	Scrum/XP	12	2 to 3
P6	USA	DEV	N.America-Europe	20 to 22	Internet Hosting	Scrum/XP	8	2
P7	USA	AC	N.America-S.America-Asia	18	Internet Domain	Scrum/XP	6	2
P8	USA	DEV	N.America-Oceania-Asia	9 to 10	Publishing	Scrum/XP	8	2
P9	USA	DEV	S.America-Europe	14	Web Search Engine	Scrum	24	2 to 3
P10	USA	SM	N.America-S.America-Asia	10 to 12	Software Platform	Scrum	8	3
P11	India	SM	N.America-Asia	13	Web Services	Scrum/XP	10	2
P12	India	DEV	N.America-Asia	12	Internet Hosting	Scrum/XP	18	2
P13	India	SM	N.America-Asia	17 to 20	Web Portal	Scrum/XP	5	2
P14	India	DEV	N.America-Asia	16 to 17	E-Commerce	Scrum/XP	36	2
P15	India	QA	N.America-Asia	16	E-Commerce	Scrum/XP	18	2
P16	India	SM	N.America-Asia	16	E-Commerce	Scrum/XP	18	2
P17	India	DEV	N.America-Asia	16	E-Commerce	Scrum/XP	18	2
P18	India	BA	Europe-Asia	8	Financial Services	Scrum/XP	12	2
P19	India	DEV	N.America-Asia	8 to 10	Insurance	Scrum	10	3
P20	Australia	MGT	Oceania-Asia	9 to 12	E-Commerce	Scrum/XP	12	2 to 3
P21	Australia	SM	N.America-Oceania	15	Financial Services	Scrum	9	2
P22	Australia	SM	Oceania-Asia	9 to 12	E Commerce	Scrum/XP	12	2 to 3
P23	India	QA	N.America-Asia	7 to 8	Power Distribution	Scrum	4	2
P24	India	AC	Europe-Asia	9	Automobile	Scrum/XP	5	2
P25	India	SM	N.America-Asia	24	Information Security	Scrum/XP	6	3
P26	India	AC	N.America-Asia	16	Healthcare	Scrum/XP	(3)	3
P27	India	SM	N.America-S.America	30	Financial Services	Scrum/XP	6	2
P28	India	MGT	N.America-Asia	20	E-Commerce	Scrum	18	3
P29	India	MGT	N.America-Asia	14	Social Networking	Scrum/XP	10	2
P30	India	AC	Europe-Asia	8 to 10	Retail	Scrum/XP	(5)	2 to 3
P31	India	AC	Europe-Asia	15 to 20	Retail	Scrum/XP	(7)	3
P32	India	MGT	Europe-Africa	12	Retail	Scrum/XP	18	2
P33	India	AC	Oceania-Europe-Asia	50	Recruitment	Scrum/XP	24	3
P34	India	AC	N.America-Asia	6 to 8	Real Estate	Scrum/XP	10	2
P35	India	AC	N.America-Asia	8	Online Payment	Scrum/XP	18	3
P36	India	QA	N.America-Asia	10 to 15	Web Services	Scrum/XP	18	2
P37	India	DEV	Europe-Asia	16	E-Commerce	Scrum/XP	4	2
P38	India	BA	N.America-Asia	28	E-Commerce	Scrum/XP	(2)	2
P39	India	AC	N.America-Asia	22 to 25	Telecommunication	Scrum/XP	6 to 7	2
P40	India	DEV	Oceania-Asia	7	Online Trading	Scrum/XP	6	2
P41	India	AC	Europe-Asia	10 to 12	Retail	Scrum/XP	(5)	2
P42	India	MGT	N.America-Asia	7	Healthcare	Scrum	(3)	4
P43	India	SM	N.America-Asia	7	E-Commerce	Scrum/XP	6	2 to 3
P44	India	PO	N.America-Asia	10 to 12	Cloud Computing	Scrum	4	2 to 3
P45	India	MGT	N.America-Asia	10	Financial Services	Scrum	8	3
P46	USA	AC	N.America-Asia	16	IT Services	Scrum	12	3
P47	USA	AC	N.America-Asia	20	Financial Services	Scrum	(4)	2
P48	USA	MGT	N.America-Europe	12	Online Media	Scrum/XP	(6)	2
P49	USA	AC	N.America-Europe	16	Telecommunication	Scrum	8	2
P50	USA	PO	Europe-Asia	8	Web Services	Scrum	30	2 to 3
P51	USA	SM	N.America-Asia	10	Web Application	Scrum/XP	9	2
P52	USA	SM	N.America-Africa	10	Web Application	Scrum	3	2
P53	USA	AC	USA-Europe-Asia	10	IT Services	Scrum	5 to 6	2
P54	USA	DEV	N.America-Asia	8	Education Services	Scrum	8	2
P55	USA	DEV	N.America-S.America	7 to 8	IT Services	Scrum	(12)	3

We conducted face-to-face, one-on-one, semi-structured interviews with participants using open-ended questions. With consent from the participants, we voice-recorded interviews in order to concentrate on the conversation during the interviews. Initially a set of guiding questions was prepared to develop a smooth conversation with the participants. The questions mainly focused on the background of the project, the challenges in Agile software development with distributed teams and the strategies that Agile teams adopt in order to overcome

those challenges. We transcribed the interviews and analysed them using *key point open coding* to explore the meaning in the data [3]. Open coding breaks down, examines, compares, conceptualises and categorises the data [65]. We gathered key points from the interview data and assigned a summary phrase, called a *code*, to each key point in order to produce a first level of abstraction. Using GT's *constant comparison* [24], we analysed each code with the codes from the same interview and those from other interviews. Codes related to a common

theme were grouped together to produce a second level of abstraction called a *concept*. Constant comparison of the codes gave rise to many fresh concepts. These concepts were also analysed using constant comparison, and concepts related to a common theme were grouped together to produce a third level of abstraction called a *category*. Figure 1 shows emergence of the category *Trust* from underlying concepts. Using a diagram to illustrate the emergence of a concept or a category is recommended in Grounded Theory [2], [22].



Fig. 1. Example of emergence of a category from underlying concepts

Several key concerns emerged from the initial analysis: *building trust, managing knowledge, improving communication, bridging cultural differences, and building team interaction*. We continually adjusted the interview questions to focus on these emergent key concerns. A GT study requires the theoretical sampling to be continued until *theoretical saturation* [30] is reached – that is when no more new concepts or categories emerge from the data, and further data collection would be a waste of time. In this study, we reached *theoretical saturation* upon completing 55 interviews with participants.

We wrote-up *theoretical memos* on the ideas about the codes, concepts and categories as they emerge throughout data analysis. A theoretical memo is the “*the theorising write-up of ideas about substantive codes and their theoretically coded relationships as they emerge during coding, collecting and analyzing data*” [26]. Once theoretical saturation was reached, we began sorting the theoretical memos according to the conceptual ideation of the memos. *Theoretical sorting* of memos is an “*essential step*” for formulating the theory for presentation or writing because “*it begins to put the fractured data back together*” [23]. The conceptually sorted memos richly explicates the research phenomenon and exhibits a strong connections between the categories and their properties [23], [25].

C. Emergence of Theory

The goal of GT is to generate a theory that accounts for a pattern of behaviour about what is going on in the area studied, which turns out to be the main concern of the participants and the research problem for the phenomenon being studied [23], [6], [44]. Our research has led us to discover a grounded

theory called “*The Theory of One Team*” which explains how a distributed team in Agile software development adopts explicit strategies for bridging spatial, temporal, and socio-cultural distances, while facing critical impact factors, in order to become one team. Different aspects of the theory has been described in different papers [11], [12], [13], [14], [15], [16]. In this paper, we discuss *trust* as an important factor for a distributed team to work together as one team.

IV. RESULTS: BUILDING TRUST

In this section, we describe how members of a distributed team build trust with one another in order to work together as become a successful Agile team.

A. Social Communication

Social communication is a form of casual communication for the purpose of socializing or getting to know one another from the professional and personal backgrounds. Many participants discussed the importance of social communication for building trust:

“*Social communication is so important in Agile [projects], and it goes a long way in building trust with the team members.*” —P20, Senior Manager.

Participants, however, reported that without frequent face-to-face interaction, members of a distributed team face difficulties to build trust and succeed as a team:

“*I think it is really hard to maintain trust, especially harder to establish trust without having face-to-face interactions. If trust is not there, it is hard to succeed as a team.*” —P4, Agile Coach.

Team members from different cultural backgrounds often faced difficulties to understand communication styles and work practices which were tied to each other’s cultures. Through social communication team members developed trust with one another:

“*The project manager in the USA had great difficulty initially to understand the way we do things. She was not comfortable with us and did not much trust us until she came down to India and engaged in social communication with the team.*” —P45, Senior Manager.

Team members developed solid relationships based on trust with the individuals whom they were working with, especially the customers. Opportunities for building trust within the team may arise when team members initially meet during project inception workshops. A team relationship based on trust may develop a strong bonding that forms the basis for effective communication while working from remote sites:

“*[Customers] need to trust that you are going to work in a professional manner and deliver value to them. Trust will lead to effective communication when you work with distributed teams. If [team members] don’t trust you, distribution doesn’t work [because] communication will be affected.*” —P11, Scrum Master.

Interaction between customers and project team members over phone or in person may develop mutual respect and understanding between them and it may contribute to develop long term professional relationships. Participants reported that solid relationships among team members have been helpful while making requests from customers and senior managers because they understood and trusted the team:

“It’s important for team members to have social interaction with product owners and key stakeholders so that they trust the team and negotiation becomes very easy.” —P34, Agile Coach.

Understanding the importance of socialising, teams allocated time for social communication before formal meetings for the entire team in order to socialise with one another. Some individuals engaged in social conversation using computer-mediated technology whenever possible during working hours. Cross-site visits, however, provided opportunities for social communication through face-to-face interaction.

B. Evidence of Expertise

Organising verbal presentations for sharing technical knowledge with one another helped to build trust within the team. Preparation for the presentations as well as doing the presentation required substantive effort from a team member. The effort and the willingness to provide information and share knowledge developed trust towards the presenters:

“We have a 15 minutes presentation once in a week for team members to share their own technical expertise with the team. It really helped them to develop trust on the presenter for sharing technical expertise with the team.” —P34, Agile Coach.

When team members participated in knowledge sharing activities, they gained trust from others who benefited from those activities. There were no hesitation on what to ask or whom to ask from because they trusted that no one would not look down on them for asking questions that may seem trivial to them. Trust developed naturally within a team when team members were willing share expertise and experience with the whole team:

“They were willing to participate in the knowledge sharing activities. They now realise that others will not belittle them for asking a question that may seem trivial to others. Knowledge sharing was happening without much stress on them.” —P41, Agile Coach.

When teams established and continually sustained trust, the communication across sites and collaboration between team members improved because team members were willing to share knowledge required for the tasks and worked together while supporting one another:

“After every knowledge sharing session, the communication certainly went up a fair bit, collaboration with client improved, and we understood better what we were developing. The main reason for these improvement is the trust [that] we developed during these sessions.” —P38, Business Analyst.

Team members often travelled across sites for the purpose of knowledge sharing and getting to know one another. When team members engaged in discussion with one another in order to acquire business knowledge or technical knowledge, the knowledge sharing activities helped to develop trust between them:

“Often somebody wants to learn technical knowledge available at one location or somebody wants to learn business knowledge from customer. When team members travel to the [remote] sites, trust develops between them and [the] individuals at the remote sites during discussions.” —P27, Scrum Master.

Besides improved communication and collaboration with one another and increased knowledge within the team, the knowledge sharing sessions helped to develop trust between team members.

C. Collaboration Initiatives

When trust existed within the team, the team members implicitly or explicitly expressed interest to collaborate with one another. Trust between team members has been an important reason for effective communication within the team. Team members kept frequent communication in order to get a clear vision of project status and to understand the tasks that have been assigned to them. Collaborative initiatives allowed them to work together as a team and build a solid team relationship based on trust:

“We knew that certain tasks have been assigned to us and we had to get them done. So, either we were helping them or they were helping us. We kept frequent communication to make sure everyone knows what we were doing. We worked well as a team [because] we had a good trusting relationship even though we have never met.” —P54, Developer.

Customers understood that development team members may need to communicate with them for different reasons, but due to time-zone differences, communication may happen outside normal working hours for either the customers or the development team members. Collaboration between development team members and customers helped to build trust between them and improved communication within the team:

“...customers absolutely trust me and give me their personal cell phone numbers and said, “Call me whenever!”. Communication for distributed teams actually gets easier as we build trust with each other while working together on the project.” —P18, Business Analyst.

When team members do not engage in conversations during meetings or do not communicate with one another, it may be difficult to build trust with them. Without effective communication between them, teams members may face difficulties to understand what is going on in the team. Some team members travelled to remote site to improve communication and build trust with one another:

“I often have to travel between both sites and sit down with them and keep frequent communication. It’s for building that trust so that we can improve communication when we get distributed.” —P44, Product Owner.

While knowledge sharing is crucial for the success of a project, some team members were not willing to share knowledge, ideas or opinions about to the project. It may be difficult to build trust when team members do not show passion for knowledge sharing or willingness to learn from one another:

“Some team members lack willingness to share ideas with the rest of the organisation, lacks personal drive to learn, [and] their passion and curiosity is not high enough.” —P20, Agile Coach.

When trust existed in the team, there was no limit for knowledge sharing between team members. The team members willingly shared all sorts of information and knowledge so that everyone may work together in order to contribute for the progress of the project:

“We took efforts to ensure that all our technical decisions we communicated upfront to team members, and we explain to them in great detail why we had taken a particular decision so that it would make more sense for them to start delivering functionality.” —P38, Business Analyst.

With short sprint cycles, team members worked together closely to achieve the sprint goals. When team members delayed the tasks assigned to them, it may be visible during daily meetings. While building trust may be difficult, the excuses for not doing project tasks may easily break trust between them:

“...some team members have the tendency to delay the work [because] they are not focused on the [project] goals. When others come to realise [that] the excuses are not valid, they break the trust.” —P30, Agile Coach.

When team members often do not attend formal meetings on time, the lack of respect and collaboration caused trust on them to decline:

“...even some team members come really late to the stand-up meetings. That’s why it was difficult to build trust within the team.” —P39, Agile Coach.

Collaboration initiatives shown within the team across sites, allowed team members to trust one another. Teams often built trust while working together at real-time.

D. Regular Communication

When team members kept frequent communication during daily meetings or retrospective meetings, they may get to know one another well and develop a sense of mutual understanding towards each other’s culture. Participants reported that promoting verbal conversation between team members may provide them with opportunities to build trust between them:

“I try to get people to know each other, facilitate meetings, and make everyone speak up during the meetings. If you can get them to talk [to each other], you can build trust and solve a lot of problems.” —P53, Agile Coach.

Customers considered team members from all the different locations to be part of one single team. Hence, customers engaged in direct interaction, often face-to-face whenever possible, with team members in order to build trust between them:

“We always have someone from client side [working] with us here so that we can talk to them anytime, especially to pick up information from us, take our queries, and go back to their counterparts [at remote sites] in order to get them resolved. The customers should talk face-to-face with us often. Only then there’ll be more trust.” —P30, Agile Coach.

Regular communications allowed team members to share knowledge and project information with one another. Through frequent discussion on project requirements, trust developed between customer and the development teams:

“We have discussion with the customers on a daily basis, and we involved the customers throughout the development phases. Customers and the development team members got to know each other and trust developed between them.” —P24, Agile Coach.

Teams invested in technical infrastructure in order to promote regular communication within the team. When teams were aware of the availability of one another, they may engage in regular communication:

“We have invested a lot [of money] for creating smart environments [where] we actually see the [remote] members online and know what exactly the team is doing right now. We can talk to them whenever we need to.” —P22, Scrum Master.

Regular communication during formal meetings and casual conversation allowed team members to build and sustain trust within the team.

E. Substantive and Honest Feedback

When team members provided substantive and honest feedback to the whole team, trust developed between them as a result of spending time and effort to give the feedback. Therefore, during daily meetings, team members attempted to provide substantive and honest feedback about to the tasks carried out for the projects to the whole team:

“During daily stand-up meetings, we’re interacting with the entire team. There’s honest feedback going back and forth from everybody in the team.” —P14, Developer.

Teams conducted retrospective meetings after each iteration to assess the level of success of the iteration against the iteration and projects goals. When the team members gave substantive and honest feedbacks, the knowledge acquired from the retrospective meetings may be useful for improving the processes and practices that are in place within the team:

“...based on the feedback from clients and team members, we’ll be taking corrective actions and analysing whether the project is going in the right way and whether it can be completed in time.” – P23, Quality Analyst.

Team members provided honest feedback with regard to team performance and remained open and transparent with one another in order to recognise the strengths and weaknesses of the entire team. With honest feedback from one another, team members sensed a feeling of trust developing within the teams:

“...we can learn from one another [and] we get feedback [about] how we work together and how is our team performance. Everyone shares their opinions and gives useful feedback. It develops the feeling of trust within the team.” – P55, Developer.

While giving honest feedback was useful for the team member to improve on the processes and practices, the team members also used the feedback to reflect on their behaviour for self-improvement in order to build a cohesive team.

F. Team above Self

During retrospective meetings, team members reflected on how to improve practices or processes in place within the team. When team members suggested good ideas or opinions for improvements, it was important that other members of the team considered to accept the suggestions for the benefit of the whole team rather than disagreeing for personal reasons. Team members develop trust on one another when they give importance to the ideas and opinions of others in the team:

“We may have our own perception of how to adopt certain practices, but we also accept ideas and opinions from other members and accept them for the best interest of the team.” —P40, Developer.

When changes were suggested within a team, the team members often engaged in discussions in order to understand the reasons for adopting the changes. During the discussions, team members considered the benefits for the whole team rather than for any individual:

“When we wanted to implement some new ideas or new practices, we always discuss them with the whole team. Though bringing new changes for the entire team will be difficult, [but] the team members always accept the changes for the benefit of the team.” —P33, Agile Coach.

Even though some team members may not agree to decisions adopted within the team, they were still willing to accept the decisions and adopted the changes that comes with the decision. Participants reported that trust developed within a team when team members understood that team’s objectives must supersede those of the individuals:

“We can work together [although] we are argumentative and opinionated, and we question each other on our decisions. Finally, what matters is the team, not the self. It all comes out from the trust that we [have] built on one another.” —P16, Scrum Master.

Team members often travelled to different sites of the team for various reasons. Due to a limited budget for travelling, the team members cautiously spent the funds so that it could benefit more team members:

“When we travel, we get the cheapest possible flight and stay in an economical hotel so that travel fund can be useful for many team members than just for the few.” —P38, Business Analyst.

When team members faced challenges in completing tasks, the team members often helped one another in order to achieve the desired goals of the project. Trust developed within a cohesive team that worked together while providing help to one another in completing the tasks for a project:

“...I knew that I wasn’t alone in doing a task. The rest of them always helped me in so many ways to get the tasks completed. Every task was [completed with] the team’s effort, not simply a single member’s effort.” —P12, Developer.

Trust developed within a cohesive team where team members gave importance to the entire team rather than themselves with regard to the successful completion of the software development project.

G. Communication with Concern

Trust may develop within the team when team members across different sites attempt to understand the challenges and difficulties faced while carrying out the software development activities. Communication with compassion and concern for the team members helped them to work closely in order to support one another to complete the tasks:

“...when we sincerely show care and concern for the team members, there will be natural trust on one another within the team [and] it gives them a good reason to work together in every task.” —P31, Agile Coach.

Even when a team member made mistakes while doing the work, the team accepted the mistake and helped him or her to learn from the mistake rather than criticising that team member. The perceptions of interpersonal concern and mutual understanding helped to develop trust within the team:

“The idea we impart is an individual can make mistakes [but] every time if it is a new mistake, it is acceptable. So, it’s [meant] to make them realise their situation without hurting them or arguing with them.” —P33, Agile Coach.

Participants reported that trust developed when team members demonstrated concern for one another. While the constraints across sites were different, team members who took extra effort to understand those constraints were able to communicate with team members with concern:

“...knowing the constraints for the team [in India], gives me empathy and sympathy for the [individuals] whom work under such conditions. It allows me to have an informed conversation with them when we have discussions with them.” —P22, Scrum Master.

Team members often engaged in casual communication in order to communicate to one another, not just for project related matters but also for personal reasons. Showing concerns during communication may develop trust within the teams.

V. RELATED WORK

While trust among team members is imperative for software development projects, substantial effort are required to build trust within a distributed team [58]. Hung et al. [39] points out the existence of high initial trust among team members in virtual teams although traditional models of trust have seen trust as being created as a result of a long history of interaction between individuals. They argue that individuals in a project team form trust attitudes through three distinct routes: *peripheral*, *central*, and *habitual*. In the initial stages of a relationship, members of a virtual team rely on peripheral cues such as third party information, social categories, roles and rules because they lack information about one other in order to form trust. Once individuals have shared history and knowledge of the other individual, they use the central route, which involves the assessment of the other individual's ability, benevolence and integrity. Finally, after long periods of shared history in which the individuals develop a habitual pattern of trust, they enact prior trust attitudes via the habitual route.

Distributed teams in Agile software development make conscious efforts to improve team interaction and build strong working relationships amongst the team members in order to allow trust to build naturally within the team [70]. McHugh et al. [50] suggest that Scrum ceremonies, particularly daily meetings, increase trust within the team by providing transparency and visibility of project status, enhancing accountability and collective responsibilities, increasing open and frequent communication, and sharing of knowledge and obtaining feedback. Our study confirms that regular communication is crucial for trust to develop between team members located across different sites. We also found that exposing expertise of team members, particularly through knowledge sharing activities, has been effective to build trust across different sites.

Regular face-to-face meetings and effective communication are important for building trust [67]. Team members appreciate initiatives that allow the customers to trust the team for long term working relationships [5]. Frequent visits by distributed partners and sponsor visits, and building a cohesive team have been contributing significantly for building trust in distributed teams [58]. Our study confirms that social communication, particularly during cross-site visits, has been useful for building trust between team members. While circumstances and constraints in across global sites can be different, we also found that trust develops when team members demonstrate concern and empathy for one another.

In another companion paper [15], we reported that lack of trust among team members leads to adverse effects for the Agile teams. We thus suggest that distributed teams not only take conscious efforts to build trust within the team, but also take every necessary measure to avoid trust to decline within the team.

VI. THREATS TO VALIDITY

A GT research study produces a 'mid-range' theory, which means that the theory can be modified with further analysis using new data from the same context. We do not claim that our findings are universally generalisable to all distributed Agile teams, but rather our findings accurately characterize the contexts studied. In GT, claims for universal generalisability is based on the fit and transferability of the results where another researcher could employ the GT's constant comparison method on new data to determine whether a substantive results fits the new data, and the results are transferable to another context [1], [21], [35].

Another common criticism of employing GT in an empirical software engineering research is regarding validation of the emergent theory [1]. Glaser [26] states that the focus of GT is the *generation* of theory, and validation of the emergent theory may be undertaken by another researcher using *different* research methods. Glaser [24], however, suggests to evaluate the credibility of an emergent theory through the following four criteria: *fit*, *relevance*, *work*, and *modifiability*. *Fit* is considered the most important criterion for evaluating validity and truth of a grounded theory, whereas the criteria *work*, *relevance*, and *modifiability* supports the 'fitness' of the theory and useful in the broader evaluation of the quality of the theory [26], [46]. Urquhart et al. [66] suggest theoretical sampling is the single most important contributor to the 'fit' of a theory. Theoretical sampling ensures the comprehensive nature of the theory – well grounded in data and explains 'what is actually going on'. Without theoretical sampling, it will be impossible to establish how 'saturated' the theory is. In this study, theoretical sampling guided us in the selection of suitable participants who provided explanations on the emerging main concerns and the evolution of interview questions that focused on the emerging concerns. We interviewed participants from *different* roles in the Agile software development, across *different* software companies in *different* countries, and we also continually evolved the interview questions in order to focus on the concerns that seem central to the emerging theory. We continued theoretical sampling until we noticed complete redundancy, or theoretical saturation, in the new data.

In addition to these evaluation criteria, this interpretive, qualitative research study adopts the *ethical* and *substantive* validation approaches of good qualitative research [4], and then creatively combines the theoretical sensitivity of the researcher with GT's rigorous analysis process to produce valuable interpretations [23]. Angen [4] argues that validation is an ethical question that must be addressed from inception to completion of research – validation must be considered when research is formulated, carried out and written up [4]. To achieve the ethical validation, we have diligently followed the processes in GT research methods from the start until the end of the research study. For example, we avoided a huge upfront literature review, used theoretical sampling for data collection and analysis until theoretical saturation was reached, continually wrote theoretical memos on the emergent

constructs, and performed theoretical sorting of the memos in order to develop a grounded theory which richly explicates the research phenomena. Angen [4] argues that the ‘chain of interpretations’ must be documented and produced to others as evidence of how the conclusions were reached in order to evaluate the trustworthiness of the meanings of the results. To achieve the substantive validation, we included selected interview quotations throughout the results section in order to document evidence of how the theory was developed in this research study.

VII. CONCLUSION AND FUTURE WORK

In this doctorate research study, we discovered a grounded theory called “*The Theory of One Team*” which explains how a distributed team in Agile software development adopts explicit strategies for bridging spatial, temporal and socio-cultural distances, while facing critical impact factors, in order to become one team. Though in general this study was not specifically an investigation into whether trust is important for distributed teams or how to build trust among members of a distributed team, a key concern of “building trust” emerged right from our initial analysis. We found that trust within distributed teams affects several other important concerns of the distributed teams such as improving communication [13], bridging cultural differences [12], building team interaction [16] and managing knowledge [14]. Further in-depth investigation has led us to discover how members of a distributed team in Agile software development build trust with one another in order to work together as one team.

Though this in-depth research study reports seven emergent techniques for building trust, we believe that there can be other subtle trust building techniques that can be useful and effective in their own contexts, but did not emerge from our analysis. We understand that building trust in a distributed team can be difficult because members of the distributed team often have little or no prior experiences working together. We found that a distributed team that has built trust steadily and systematically over time could address impediments and problems reliably, quickly and efficiently, and establish a long term working relationship between customers and development team members across global sites.

The understanding of the importance of trust for distributed teams gained from this study could be used as a foundation for conducting future studies. We suggest that our study is replicated within a different context, such as involving an outsourced team with members from different organisations. Another avenue for future study is to measure the findings from this study using a quantitative research method in order to investigate how many teams adopt the different techniques presented in this paper for building trust.

VIII. ACKNOWLEDGMENTS

Thanks to the participants of this study. Thanks to Dr. Petra Malik and Dr. George Allan for their help.

REFERENCES

- [1] S. Adolph, W. Hall, and P. Kruchten. A methodological leg to stand on: Lessons learned using grounded theory to study software development. In *Proceedings of the 2008 Conference of the Center for Advanced Studies on Collaborative Research*, pages 166–178, New York, NY, USA, 2008. ACM.
- [2] G. Allan. A critique of using grounded theory as a research method. *Electronic Journal of Business Research Methods*, 2(1):1–10, 2003.
- [3] G. Allan. The use of grounded theory as a research method: warts all. In *European Conference on Research Methodology for Business and Management Studies*, pages 9–19. MCIL, 2005.
- [4] M. J. Angen. Evaluating interpretive inquiry: Reviewing the validity debate and opening the dialogue. *Qualitative Health Research*, 10(3):378–395, 2000.
- [5] M. A. Babar, J. M. Verner, and P. T. Nguyen. Establishing and maintaining trust in software outsourcing relationships: An empirical investigation. *Journal of Systems and Software*, 80(9):1438–1449, 2007.
- [6] K. Backman and H. A. Kyngäs. Challenges of the grounded theory approach to a novice researcher. *Nursing & Health Sciences*, 1(3):147–153, 1999.
- [7] D. Bandow. Time to create sound teamwork. *Journal of Quality and Participation*, 24(2):41–47, 2001.
- [8] K. Charmaz. *Constructing Grounded Theory: A Practical Guide Through Qualitative Analysis*. Sage Publications, Thousand Oaks, CA, USA, 2006.
- [9] G. Coleman and R. O’Connor. Using grounded theory to understand software process improvement: A study of Irish software product companies. *Information and Software Technology*, 49(6):654–667, 2007.
- [10] R. Davies and L. Sedley. *Agile Coaching*. Pragmatic Bookshelf, USA, 1 edition, 2009.
- [11] S. Dorairaj, J. Noble, and P. Malik. Understanding the importance of trust in distributed Agile projects: A practical perspective. In *Agile Processes in Software Engineering and Extreme Programming*, volume 48 of *Lecture Notes in Business Information Processing*, pages 172–177. Springer Berlin Heidelberg, Trondheim, Norway, 2010.
- [12] S. Dorairaj, J. Noble, and P. Malik. Bridging cultural differences: A grounded theory perspective. In *Proceedings of the 4th India Software Engineering Conference*, pages 3–10. ACM, 2011.
- [13] S. Dorairaj, J. Noble, and P. Malik. Effective communication in distributed Agile software development teams. In *Agile Processes in Software Engineering and Extreme Programming*, volume 49 of *Lecture Notes in Business Information Processing*, pages 102–116. Springer Berlin Heidelberg, Madrid, Spain, 2011.
- [14] S. Dorairaj, J. Noble, and P. Malik. Knowledge management in distributed agile software development. In *Agile Conference (AGILE)*, 2012, pages 64–73. IEEE, 2012.
- [15] S. Dorairaj, J. Noble, and P. Malik. Understanding lack of trust in distributed Agile teams: A grounded theory study. In *Evaluation & Assessment in Software Engineering (EASE 2012)*, 16th International Conference on, pages 81–90. IET, 2012.
- [16] S. Dorairaj, J. Noble, and P. Malik. Understanding team dynamics in distributed Agile software development. *Agile Processes in Software Engineering and Extreme Programming*, pages 47–61, 2012.
- [17] T. Dybå and T. Dingsøyr. Empirical studies of Agile software development: A systematic review. *Information Software Technology*, 50(9–10):833–859, 2008.
- [18] M. Fowler and J. Highsmith. The Agile Manifesto. *Software Development*, 9(8):28–35, 2001.
- [19] D. Fox, J. Sillito, and F. Maurer. Agile methods and user-centered design: How these two methodologies are being successfully integrated in industry. In *Proceedings of AGILE 2008 Conference*, pages 63–72, 2008.
- [20] D. Gambetta. *Trust: Making and Breaking Cooperative Relations*. Blackwell Publishers, 1990.
- [21] S. Gasson. Rigor in grounded theory research: An interpretive perspective on generating theory from qualitative field studies. In M. Whitman and A. Wozniakowski, editors, *Handbook for Information Systems Research*, pages 79–102. Idea Group Publishing, Hershey, PA, 2003.
- [22] S. Georgieva and G. Allan. Best practices in project management through a grounded theory lens. *Electronic Journal of Business Research Methods*, 6(1):43–52, 2008.
- [23] B. Glaser. *Theoretical Sensitivity: Advances in Methodology of Grounded Theory*. Sociology Press, Mill Valley, CA, 1978.

- [24] B. G. Glaser. The constant comparative method of qualitative analysis. *Social Problems*, 12(4):436–445, 1965.
- [25] B. G. Glaser. *Basics of Grounded Theory Analysis: Emergence vs Forcing*. Sociology Press, Mill Valley, CA, 1992.
- [26] B. G. Glaser. *Doing Grounded Theory: Issues and Discussions*. Sociology Press, Mill Valley, CA, 1998.
- [27] B. G. Glaser. *The Grounded Theory Perspective: Conceptualization Contrasted With Description*. Sociology Press, Mill Valley, CA, 2001.
- [28] B. G. Glaser. *The Grounded Theory Perspective III: Theoretical Coding*. Sociology Press, Mill Valley, CA, 2005.
- [29] B. G. Glaser. *Doing Formal Grounded Theory: A Proposal*. Sociology Press, Mill Valley, CA, 2007.
- [30] B. G. Glaser and A. L. Strauss. *The Discovery of Grounded Theory: Strategies for Qualitative Research*. Sociology Press, Aldine, Chicago, 1967.
- [31] C. Goulding. Grounded theory: some reflections on paradigm, procedures and misconceptions. Working Paper WP006/99, University of Wolverhampton, UK, June 1999.
- [32] M. A. Hall. Researching medical trust in the United States. *Journal of Health Organization and Management*, 20:456–467, 2006.
- [33] E. Hasnain and T. Hall. Investigating the role of trust in Agile methods using a light weight systematic literature review. In P. Abrahamsson, R. Baskerville, K. Conboy, B. Fitzgerald, L. Morgan, X. Wang, W. Aalst, J. Mylopoulos, M. Rosemann, M. J. Shaw, and C. Szyperski, editors, *Agile Processes in Software Engineering and Extreme Programming*, volume 9 of *Lecture Notes in Business Information Processing*, pages 204–207. Springer Berlin Heidelberg, 2008.
- [34] J. Highsmith. *Agile Project Management*. Addison Wesley, 2004.
- [35] R. Hoda, J. Noble, and S. Marshall. Developing a grounded theory to explain the practices of self-organizing Agile teams. *Empirical Software Engineering*, pages 1–31.
- [36] R. Hoda, J. Noble, and S. Marshall. Organizing self-organizing teams. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering*, pages 285–294, New York, USA, 2010.
- [37] H. Holmstrom, E. O. Conchuir, P. J. Agerfalk, and B. Fitzgerald. Global software development challenges: A case study on temporal, geographical and socio-cultural distance. *International Conference on Global Software Engineering*, pages 3–11, 2006.
- [38] H. Holmstrom, B. Fitzgerald, P. J. Agerfalk, and E. O. Conchuir. Agile practices reduce distance in global software development. *Information Systems Management*, 23(3):7–18, 2006.
- [39] Y. Hung, A. Dennis, and L. Robert. Trust in virtual teams: Towards an integrative model of trust formation. In *System Sciences, 2004. Proceedings of the 37th Annual Hawaii International Conference on*, pages 1–11, 2004.
- [40] S. L. Jarvenpaa, T. R. Shaw, and D. Staples. Toward contextualized theories of trust: The role of trust in global virtual teams. *Information Systems Research*, 15, 2004.
- [41] M. Korkala and P. Abrahamsson. Communication in distributed Agile development: A case study. In *33rd EUROMICRO Conference on Software Engineering and Advanced Applications*, pages 203–210, 2007.
- [42] M. Korkala, M. Pikkarainen, and K. Conboy. Distributed Agile development: A case study of customer communication challenges. In *Agile Processes in Software Engineering and Extreme Programming*, volume 31 of *Lecture Notes in Business Information Processing*, pages 161–167. Springer Berlin Heidelberg, 2009.
- [43] M. C. Lander, R. L. Purvis, G. E. McCray, and W. Leigh. Trust-building mechanisms utilized in outsourced IS development projects: A case study. *Information & Management*, 41(4):509–528, 2004.
- [44] H. Lehmann. The dynamics of international information systems: Anatomy of a grounded theory investigation. In R. Sharda and S. VoB, editors, *Integrated Series in Information Systems*, volume 23. Springer, New York, NY, USA, 2010.
- [45] P. Lencioni. *The Five Dysfunctions of a Team*. Jossey-Bass, San Francisco, USA, 1 edition, 2002.
- [46] K. Lomborg and M. Kirkevold. Truth and validity in grounded theory: a reconsidered realist interpretation of the criteria: Fit, work, relevance and modifiability. *Nursing Philosophy*, 4(3):189–200, 2003.
- [47] S. P. Marsh. *Formalising Trust as a Computational Concept*. PhD thesis, University of Stirling, Scotland, UK, 1994.
- [48] A. Martin, R. Biddle, and J. Noble. The XP customer team: A grounded theory. In *Proceedings of AGILE 2009 Conference*, pages 57–64, 2009.
- [49] R. C. Mayer, J. H. Davis, and F. D. Schoorman. An integrative model of organizational trust. *The Academy of Management Review*, 20(3):709–734, 1995.
- [50] O. McHugh, K. Conboy, and M. Lang. The impact of Agile practices on trust in software project teams. *IEEE Software*, PP(99):1, 2011.
- [51] O. McHugh, K. Conboy, and M. Lang. Using Agile practices to build trust in an Agile team: A case study. In J. Pokorny, V. Repa, K. Richta, W. Wojtkowski, H. Linger, C. Barry, and M. Lang, editors, *Information Systems Development*, pages 503–516. Springer New York, 2011.
- [52] N. B. Moe and D. Smite. Understanding lacking trust in global software teams: A multi-case study. In *Product-Focused Software Process Improvement*, volume 4589, pages 20–34. Springer Berlin / Heidelberg, 2007.
- [53] A. Moghaddam. Coding issues in grounded theory. *Issues In Educational Research*, 16:52–66, 2006.
- [54] S. Nerur, R. Mahapatra, and G. Mangalaraj. Challenges of migrating to Agile methodologies. *Communications of ACM*, 48:72–78, May 2005.
- [55] N. V. Oza, T. Hall, A. Rainer, and S. Grey. Trust in software outsourcing relationships: An empirical investigation of Indian software companies. *Information and Software Technology*, 48(5):345–354, 2006.
- [56] G. Piccoli and B. Ives. Trust and the unintended effects of behavior control in virtual teams. *MIS Quarterly*, 27(3):365–395, 2003.
- [57] A. J. Pickard. *Research Methods in Information*. Facet Publishing, London, UK, 2007.
- [58] B. Ramesh, L. Cao, K. Mohan, and P. Xu. Can distributed software development be Agile? *Communication of the ACM*, 49(10):41–46, 2006.
- [59] J. K. Rempel, J. G. Holmes, and M. P. Zanna. Trust in close relationships. *Journal of Personality and Social Psychology*, 49:95–112, 1985.
- [60] H. Robinson and H. Sharp. The characteristics of XP teams. In J. Eckstein and H. Baumeister, editors, *Extreme Programming and Agile Processes in Software Engineering*, Lecture Notes in Computer Science, pages 139–147. Springer, 2004.
- [61] R. Sabherwal. The role of trust in outsourced IS development projects. *Communications of ACM*, 42:80–86, February 1999.
- [62] R. S. Schreiber and P. N. Stern. *Using Grounded Theory in Nursing*. Springer Publishing, Broadway, New York, 2001.
- [63] K. Schwaber and M. Beedle. *Agile Software Development with Scrum*. Prentice Hall PTR, Upper Saddle River, NJ, USA, 2001.
- [64] H. Sharp and H. Robinson. Collaboration and coordination in mature eXtreme programming teams. *International Journal of Human-Computer Studies*, 66(7):506–518, 2008. Collaborative and Social Aspects of Software Development.
- [65] A. Strauss and J. Corbin. *Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory*. Sage Publications, Thousand Oaks, CA, USA, 1998.
- [66] C. Urquhart, H. Lehmann, and M. D. Myers. Putting the ‘theory’ back into grounded theory: guidelines for grounded theory studies in information systems. *Information Systems Journal*, 20(4):357–381, 2010.
- [67] M. Vax and S. Michaud. Distributed Agile: Growing a practice together. In *Proceedings of AGILE 2008 Conference*, pages 310–314, Los Alamitos, CA, USA, 2008. IEEE Computer Society.
- [68] E. Whitworth and R. Biddle. The social nature of Agile teams. In *Proceedings of AGILE 2007 Conference*, pages 26–36, Washington, DC, USA, 2007. IEEE Computer Society.
- [69] J. M. Wilson, S. G. Straus, and B. McEvily. All in due time: The development of trust in computer-mediated and face-to-face teams. *Organizational Behavior and Human Decision Processes*, 99(1):16–33, 2006.
- [70] C. Young and H. Terashima. How did we adapt Agile processes to our distributed development? In *Proceedings of AGILE 2008 Conference*, pages 304–309, Los Alamitos, CA, USA, 2008. IEEE Computer Society.